

Лабораторная работа №5

Аппарат сетей Петри

Малюга Валерия Васильевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Общая информация	7
3.2	Обедающие философы	8
4	Выполнение лабораторной работы	10
4.1	Выполнение предложенных скриптов	10
4.2	Преобразование в литературный код	16
4.3	Преобразование в производные форматы	19
4.4	Реализия скриптов	20
4.5	Анализ	29
5	Выводы	31
	Список литературы	33

Список иллюстраций

4.1	Выполнение скрипта <code>dining_philosophers.jl</code>	10
4.2	Полученные графики после выполнения <code>dining_philosophers.jl</code> .	11
4.3	График классической модели	12
4.4	График классической модели	13
4.5	Таблица <code>dining_classic</code>	14
4.6	Таблица <code>dining_arbiter</code>	14
4.7	Выполнение скрипта <code>dining_philosophers_animation.jl</code>	14
4.8	Анимация <code>philosophers_simulation.gif</code>	15
4.9	Финальный график	16
4.10	Преобразование в литературный код <code>dining_philosophers.jl</code>	17
4.11	Преобразование в литературный код <code>dining_philosophers_animation.jl</code>	17
4.12	Преобразование в литературный код <code>dining_philosophers_report.jl</code>	18
4.13	Каталог скриптов проекта	18
4.14	Получение производных форматов	19
4.15	Ноутбук <code>dining_philosophers_lit</code>	19

Список таблиц

1 Цель работы

1. Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
2. Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую.
3. Провести имитационное моделирование (стохастическое и детерминированное) и выявить наличие deadlock.
4. Модифицировать сеть, чтобы предотвратить deadlock.
5. Проанализировать результаты и оформить отчёт с графиками и анимацией.

2 Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.
- Выполнить дополнительные задания.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.
- Результирующие файлы выложить на git.

3 Теоретическое введение

3.1 Общая информация

Сеть Петри была предложена Карлом Адамом Петри в 1962 году [3]. Это математический аппарат для моделирования дискретных систем. Графически она представляется как двудольный ориентированный граф двух типов вершин: позиции (круги) и переходы (прямоугольники).

- Позиции (Places) суть пассивные элементы, описывающие состояние системы (например, наличие ресурса или выполнение условия). Внутри позиции могут находиться фишки (tokens) — неотрицательное целое число, указывающее на количество ресурсов.
- Переходы (Transitions) суть активные элементы, описывающие события или действия системы. Они могут срабатывать, изменяя состояние модели.
- Дуги (Arcs) суть направленные соединения между позициями и переходами (но не между двумя позициями или двумя переходами), которые показывают, как состояние влияет на события и наоборот.
- Маркировка (Marking) есть распределение фишек по позициям в определённый момент времени, то есть текущее состояние модели. Смена маркировок происходит при срабатывании переходов в соответствии с определёнными правилами. Теория сетей Петри, появившаяся как инструмент для анализа химических процессов, сегодня является мощным и наглядным математическим аппаратом. Она незаменима везде, где нужно описать параллельные, асинхронные

и распределённые системы. В её основе лежат всего четыре элемента, а богатство поведения возникает из их комбинации.

3.1.1 Формальное определение

Сеть Петри (N-схема) — это четвёрка: $N = (P, T, F, M_0)$, где: - $P = p_1, p_2, \dots, p_n$ — конечное множество позиций (мест); - $T = t_1, t_2, \dots, t_m$ — конечное множество переходов; - $F \subseteq (P \times T) \cup (T \times P)$ — множество дуг (отношения инцидентности); - $M_0 \square P \rightarrow \mathbb{N}$ — маркировка, задающая начальное распределение фишек.

3.2 Обедающие философы

3.2.1 Классическая постановка

За круглым столом сидят N философов (обычно 5). Перед каждым философом стоит тарелка с едой. Между каждыми двумя соседними философами лежит одна вилка (или палочка для еды). Таким образом, количество вилок равно количеству философов. Задача была сформулирована Эдсгером Дейкстрой в 1965 году как иллюстрация проблемы синхронизации в параллельных вычислительных системах [2].

3.2.2 Основные положения

1. Правила поведения философов:

- Философ может находиться в одном из трёх состояний: думает, голоден (хочет есть), ест.
- Чтобы поесть, философу необходимы две вилки — та, что слева от него, и та, что справа.
- Если философ не может получить обе вилки одновременно, он ждёт, пока они освободятся.

- Поев, философ кладёт обе вилки обратно на стол и возвращается к размышлениям.

2. Ограничения:

- Вилками нельзя пользоваться одновременно двум философам.
- Философ не может отнять вилку у соседа — только дожидаться, пока тот её положит.

3.2.3 Проблема взаимной блокировки (deadlock)

При наивной реализации (каждый философ сначала берёт левую вилку, затем правую) возникает тупиковая ситуация: если все философы одновременно возьмут левую вилку, то каждый будет ждать правую, которая уже занята соседом. В результате никто не может поест — система замирает. Это и есть взаимная блокировка, подробно описанная в [1].

Для предотвращения deadlock предложены различные подходы:

- Арбитр (официант) — вводится дополнительный процесс, который разрешает есть не более чем $N-1$ философам одновременно.
- Асимметричное правило — философы с нечётным номером сначала берут левую вилку, затем правую, а чётные — наоборот.
- Атомарный захват — философ берёт обе вилки одновременно (если они свободны) иначе не берёт ни одной.
- Использование мьютексов и условных переменных в программных реализациях.

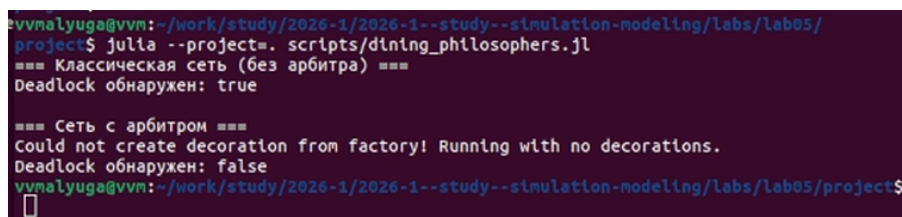
4 Выполнение лабораторной работы

4.1 Выполнение предложенных скриптов

Были созданы и выполнены все предложенные скрипты.

При выполнении скрипта `dining_philosophers.jl` были получены:

1. Анализ (подтвердилось, что при моделировании классической задачи обедающих философов происходит блокировка, а при модифицированной с арбитром блокировки нет (рис. 4.1)).



```
vvnalyuga@vvm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project$ julia --project=. scripts/dining_philosophers.jl
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true

=== Сеть с арбитром ===
Could not create decoration from factory! Running with no decorations.
Deadlock обнаружен: false
vvnalyuga@vvm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project$
```

Рисунок 4.1: Выполнение скрипта `dining_philosophers.jl`

2. Два графика, на которых видны изменения маркировок во времени по обеим моделям (рис. 4.2): классической и с арбитром. Видно, что в классической модели (рис. 4.3) спустя время все философы становятся голодными и никто не ест: происходит блокировка. В модели с арбитром (рис. 4.4) видно, как постоянно меняются состояния у философов и блокировка не происходит.

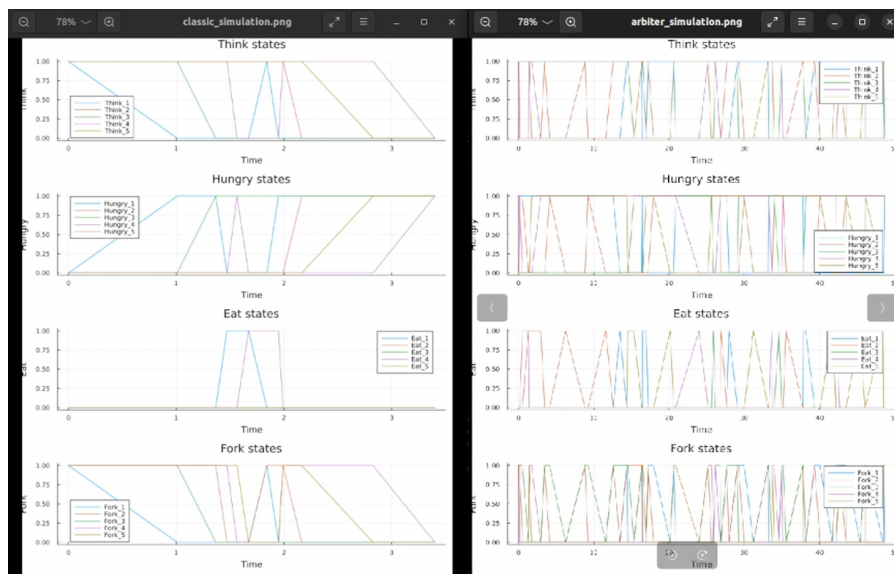


Рисунок 4.2: Полученные графики после выполнения `dining_philosophers.jl`

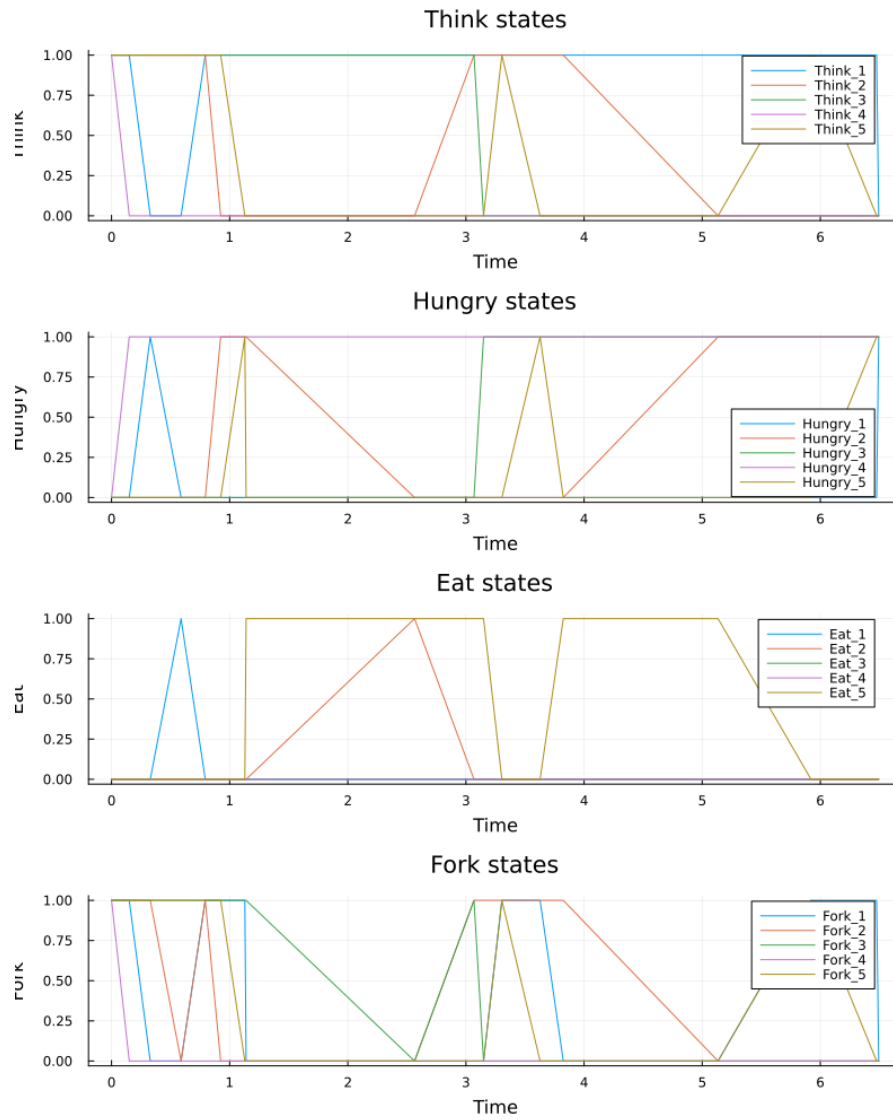


Рисунок 4.3: График классической модели

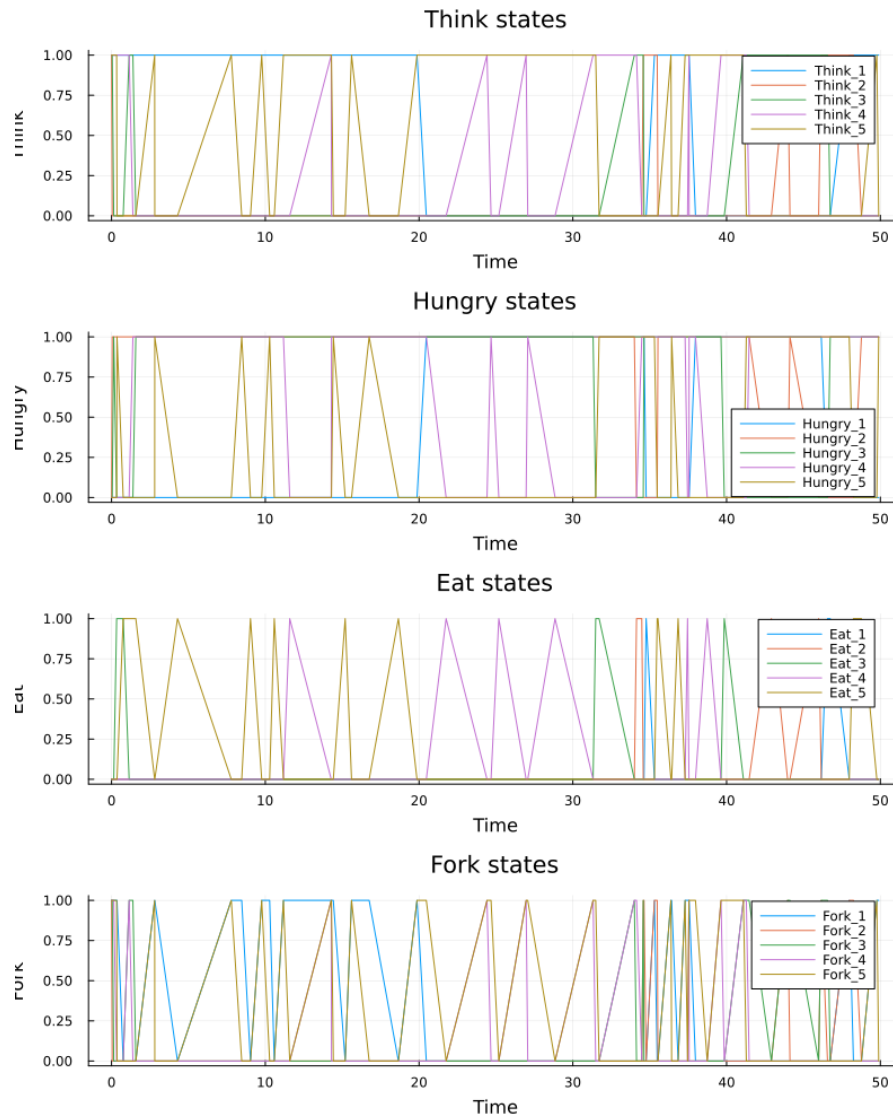


Рисунок 4.4: График классической модели

- Две таблицы `dining_classic` и `dining_arbiter`, в которых отображены изменения маркировок состояний для каждого философа (0 или 1) в течение времени(рис. 4.5, рис. 4.6).

dining_classic.csv - LibreOffice Calc

FileEditViewInsertFormatStylesSheetDataToolsWindowHelp

Рисунок 4.5: Таблица dining_classic

A1	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
time	Think_1	Think_2	Think_3	Think_4	Think_5	Hungry_1	Hungry_2	Hungry_3	Hungry_4	Hungry_5	Eat_1	Eat_2	Eat_3	Eat_4	Eat_5	Fork_1	Fork_2	Fork_3	Fork_4	Fork_5	Arbiter
1	0	1	1	1	1	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	4
2	0.00058529682041846	0	1	1	1	1	1	0	0	0	0	0	0	0	0	0	1	1	1	1	3
3	0.0534478373245496	0	1	1	0	1	1	0	0	1	0	0	0	0	0	0	1	1	0	1	2
4	0.138011796749255	0	0	1	0	1	1	1	0	1	0	0	0	0	0	0	0	1	1	0	1
5	0.551032864784172	0	0	1	0	1	1	1	0	0	0	0	0	0	1	0	0	0	1	0	1
6	1.34762256888817	0	0	1	0	1	1	0	0	0	0	1	0	1	0	0	0	0	0	0	1
7	1.43487171317109	0	0	1	1	1	1	0	0	0	0	1	0	1	0	0	0	0	0	0	1
8	1.76238022255097	0	0	1	1	0	1	0	0	0	1	0	1	0	0	0	0	0	1	0	1
9	2.97291834237727	0	0	1	0	0	1	0	0	1	1	0	1	0	0	0	0	0	0	0	0
10	3.4090682968336	0	1	1	0	0	1	0	0	1	1	0	0	0	0	0	0	1	1	0	1
11	4.15603134877793	0	0	1	0	0	1	1	0	1	1	0	0	0	0	0	0	1	0	0	0
12	6.2889747253115	0	0	1	0	0	1	0	0	1	1	0	1	0	0	0	0	0	0	0	0
13	8.8552296113248	0	1	1	0	0	1	0	0	1	1	0	0	0	0	0	0	1	1	0	1
14	9.2513409870304	0	0	1	0	0	1	1	0	1	1	0	0	0	0	0	0	1	0	0	0
15	11.5833879440406	0	0	1	0	0	1	0	0	1	1	0	1	0	0	0	0	0	0	0	0
16	12.5909620825272	0	1	1	0	0	1	0	0	1	1	0	0	0	0	0	0	1	1	0	1
17	13.4788384701178	0	1	1	0	0	0	0	0	1	1	0	0	0	0	0	0	1	1	0	0
18	14.388972256422	1	1	1	0	0	0	0	0	1	1	0	0	0	0	0	1	1	1	0	2
19	14.5279448966601	1	1	1	0	0	0	0	0	1	0	0	0	0	0	1	0	1	1	0	2
20	15.3694757840469	1	1	0	0	0	0	0	0	1	0	0	0	0	0	1	0	1	0	0	1
21	16.3819068598171	1	1	0	0	1	0	0	1	1	0	0	0	0	0	1	1	0	0	1	2
22	16.4566341690243	0	1	0	0	1	1	0	1	1	0	0	0	0	0	0	1	0	0	1	1
23	16.4900068475087	0	1	0	0	0	0	0	1	1	0	0	0	0	0	0	1	0	0	0	0
24	16.972403175083	0	1	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0
25	17.2267382990687	1	1	0	0	0	0	0	1	1	1	0	0	0	0	0	1	1	0	0	1
26	17.9502036272015	1	0	0	0	0	0	0	1	1	1	0	0	0	0	0	1	0	0	0	0
27	20.15311145952926	1	0	0	0	0	0	1	1	1	0	0	0	0	0	1	0	0	0	0	0
28	20.6153210307667	1	0	0	0	1	0	1	1	1	0	0	0	0	0	1	0	0	0	1	1
29	20.8806904133918	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	0	0	0	1	0
30	22.8712597128553	0	0	0	0	1	1	1	1	0	0	0	0	0	1	0	0	0	0	0	0
31	25.1757978844352	0	0	0	1	1	1	1	1	0	0	0	0	0	0	0	0	0	1	1	1
32	25.6715907333948	0	0	0	1	0	1	1	1	0	1	0	0	0	0	0	0	0	1	0	0
33	25.8952633598951	0	0	0	1	0	1	1	1	0	1	0	0	1	0	0	0	0	0	0	0
34																					

Рисунок 4.6: Таблица dining_arbiter

При выполнении скрипта `dining_philosophers_animation.jl` (рис. 4.7) была получена анимация изменения состояний в классической модели с тремя философами (рис. 4.8).

```
vvnalyuga@vvm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project
julia --project=. scripts/dining_philosophers_report.jl
Отчёт сохранён в plots/final_report.png
Could not create decoration from factory! Running with no decorations.
vvnalyuga@vvm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project
```

Рисунок 4.7: Выполнение скрипта dining_philosophers_animation.jl

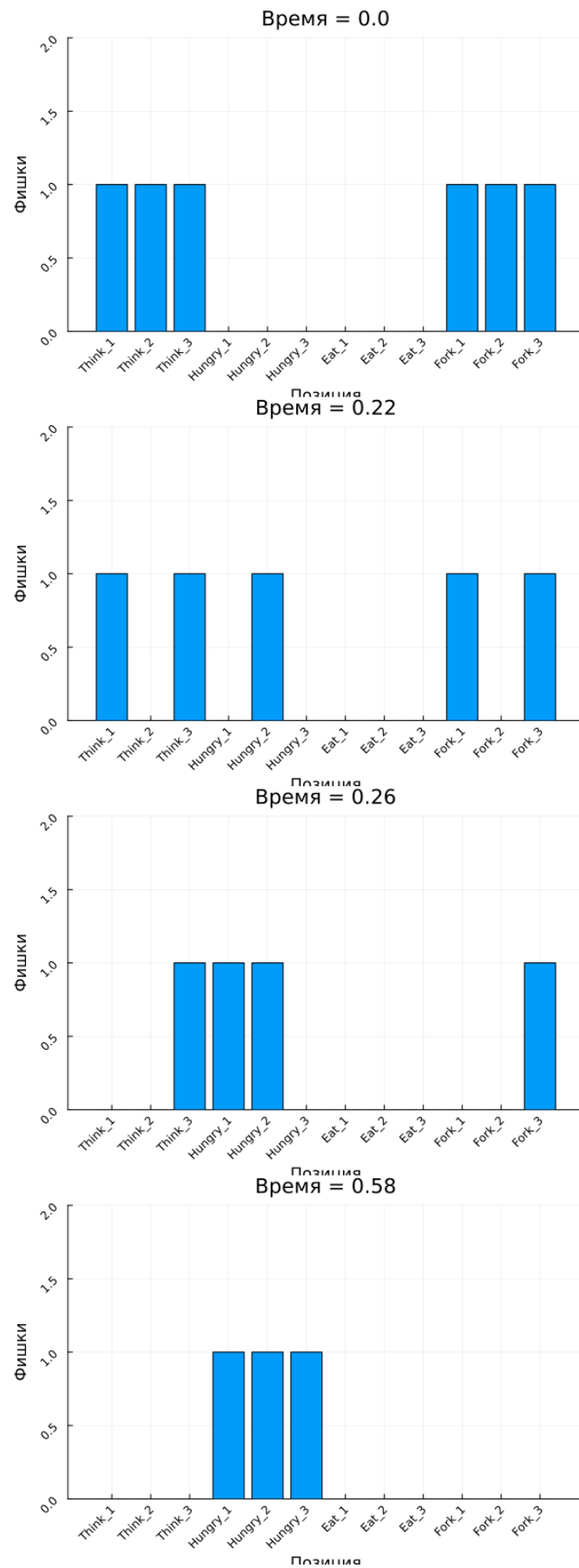


Рисунок 4.8: Анимация philosophers_simulation.gif

При выполнении скрипта `dining_philosophers_report.jl`(рис. 4.9) был получен график, на котором изображено изменение маркировки Ест во времени для обеих моделей: классической и с арбитром(рис. 4.10).

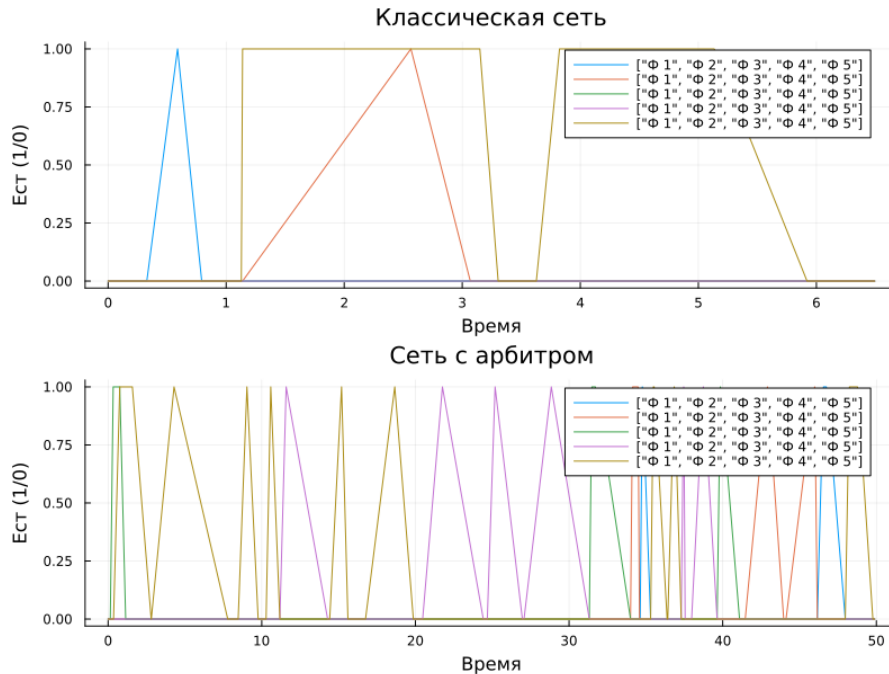


Рисунок 4.9: Финальный график

4.2 Преобразование в литературный код

Предложенные скрипты были преобразованы в литературный код(рис. 4.10, рис. 4.11, рис. 4.12).


```

Open  ~ /work/study/2026-1/2026-1-study-sim...ion-modeling/labs/lab05/project/scripts  Save
*dining_philosophers.jl  dining_philosophers_animation.jl  dining_philosophers_report.jl

1 # ### Моделирование сети Петри
2 #
3 # ### Цель: Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
4 # ### Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт
   вторую.
5 # ### Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри:
6 # ### классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock);
7 # ### модифицированная модель с арбитром, которая должна предотвращать deadlock.
8
9 using DrWatson
10 @quickactivate "project"
11 include(srcdir("DiningPhilosophers.jl"))
12 using .DiningPhilosophers
13 using DataFrames, CSV, Plots
14
15 N = 5
16 tmax = 50.0
17
18 println("=== Классическая сеть (без арбитра) ===")
19 net_classic, u0_classic, _ = build_classical_network(N)
20 df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
21 CSV.write(datadir("dining_classic.csv"), df_classic)
22 dead = detect_deadlock(df_classic, net_classic)
23 println("Deadlock обнаружен: $dead")
24 plot_classic = plot_marking_evolution(df_classic, N)
25 savefig(plotsdir("classic_simulation.png"))
26
27 println("\n=== Сеть с арбитром ===")
28 net_arb, u0_arb, _ = build_arbiter_network(N)
29 df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
30 CSV.write(datadir("dining_arbiter.csv"), df_arb)
31 dead_arb = detect_deadlock(df_arb, net_arb)
32 println("Deadlock обнаружен: $dead_arb")
33 plot_arb = plot_marking_evolution(df_arb, N)
34 savefig(plotsdir("arbiter_simulation.png"))

```

Рисунок 4.10: Преобразование в литературный код dining_philosophers.jl

```

Open  ~ /work/study/2026-1/2026-1-study-sim...ion-modeling/labs/lab05/project/scripts  Save
*dining_philosophers.jl  *dining_philosophers_animation.jl  *dining_philosophers_report.jl

1 # ### Анимация сети Петри
2 #
3 # ### Наглядная демонстрация динамики работы сети Петри во времени.
4 # ### Анимация позволяет увидеть, как меняется маркировка (фишки) в каждой позиции, и особенно наглядно
   показывает возникновение deadlock в классической модели.
5
6
7 using DrWatson
8 @quickactivate "project"
9 include(srcdir("DiningPhilosophers.jl"))
10 using .DiningPhilosophers
11 using Plots, Random
12
13 N = 3
14 tmax = 30.0
15 net, u0, names = build_classical_network(N)
16
17 Random.seed!(123)
18 df = simulate_stochastic(net, u0, tmax)
19
20 anim = @animate for row in eachrow(df)
21     u = [row[col] for col in propertynames(row) if col != :time]
22     bar(
23         1:length(u),
24         u,
25         legend = false,
26         ylims = (0, maximum(u0) + 1),
27         xlabel = "Позиция",
28         ylabel = "Фишки",
29         title = "Время = $(round(row.time, digits=2))",
30     )
31     xticks!(1:length(u), string.(names), rotation = 45)
32 end
33
34 gif(anim, plotsdir("philosophers_simulation.gif"), fps = 2)
35 println("Анимация сохранена в plots/philosophers_simulation.gif")

```

Рисунок 4.11: Преобразование в литературный код dining_philosophers_animation.jl

```

1 # ### Анимация сети Петри
2 #
3 # ### Сравнительный анализ двух моделей (с арбитром и без) по одному ключевому показателю – числу
  философов, находящихся в состоянии «Ест» (Eat_i).
4 #
5 # ### Скрипт генерирует сводный график.
6
7
8 using DrWatson
9 @quickactivate "project"
10 using DataFrames, CSV, Plots
11
12 df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
13 df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)
14 N = 5
15
16
17 eat_cols = [Symbol("Eat_$(i)") for i = 1:N] # Столбцы для состояния "Ест"
18
19 p1 = plot(
20     df_classic.time,
21     Matrix(df_classic[:, eat_cols]),
22     label = [" $\phi_i$ " for i = 1:N],
23     xlabel = "Время",
24     ylabel = "Ест ( $1/\theta$ )",
25     title = "Классическая сеть",
26 )
27 p2 = plot(
28     df_arbiter.time,
29     Matrix(df_arbiter[:, eat_cols]),
30     label = [" $\phi_i$ " for i = 1:N],
31     xlabel = "Время",
32     ylabel = "Ест ( $1/\theta$ )",
33     title = "Сеть с арбитром",
34 )
35 p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
36 savefig(plotsdir("final_report.png"))
37
38 println("Отчёт сохранён в plots/final_report.png")

```

Рисунок 4.12: Преобразование в литературный код dining_philosophers_report.jl

В итоге получила каталог с необходимыми скриптами (рис. 4.13).

Name	Size	Modified
dining_philosophers.jl	1,2 kB	22
dining_philosophers_animation.jl	795 bytes	22
dining_philosophers_animation_lit.jl	1,3 kB	22
dining_philosophers_lit.jl	1,8 kB	22
dining_philosophers_report.jl	880 bytes	22
dining_philosophers_report_lit.jl	1,3 kB	22
intro.jl	405 bytes	19
tangle.jl	2,0 kB	20 м

Рисунок 4.13: Каталог скриптов проекта

4.3 Преобразование в производные форматы

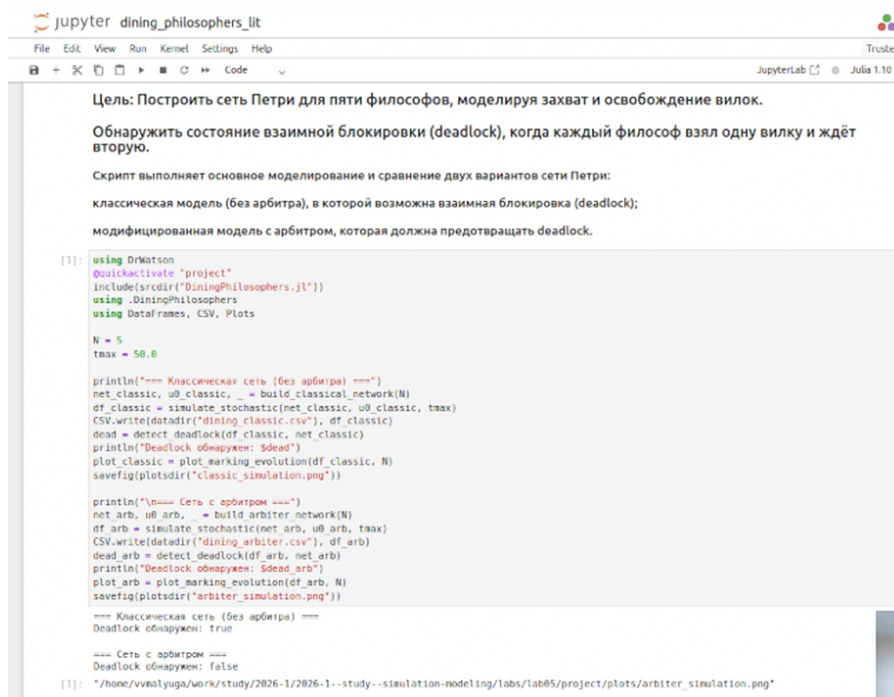
После преобразования в литературный код из скриптов были получены производные форматы с помощью скрипта `tangle.jl` (рис. 4.14).

```
getty@lab05:~/project/scripts/tangle.jl$
vvmalyuga@vwm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project$
julia --project=. scripts/tangle.jl scripts/dining_philosophers_lit.jl
Генерация из: scripts/dining_philosophers_lit.jl
[ Info: generating plain script file from `~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project/scripts/dining_philosophers_lit.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project/scripts/dining_philosophers_lit/dining_philosophers_lit.jl`
✓ Чистый скрипт: /home/vvmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project/scripts/../../project/scripts/dining_philosophers_lit/dining_philosophers_lit.jl
[ Info: generating markdown page from `~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project/scripts/dining_philosophers_lit.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project/markdown/dining_philosophers_lit/dining_philosophers_lit.qmd`
✓ Quarto: /home/vvmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project/scripts/../../project/markdown/dining_philosophers_lit/dining_philosophers_lit.qmd
[ Info: generating notebook from `~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project/scripts/dining_philosophers_lit.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project/notebooks/dining_philosophers_lit/dining_philosophers_lit.ipynb`
✓ Notebook: /home/vvmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project/scripts/../../project/notebooks/dining_philosophers_lit/dining_philosophers_lit.ipynb

Готово! Все файлы созданы.
```

Рисунок 4.14: Получение производных форматов

Были проверены полученные ноутбуки на наличие ошибок (рис. 4.15).



```
jupyter dining_philosophers_lit
File Edit View Run Kernel Settings Help
JupyterLab Julia 1.10

Цель: Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую.
Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри:
классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock);
модифицированная модель с арбитром, которая должна предотвращать deadlock.

[1]: using DrWatson
      @quickactivate "project"
      include(srcdir("DiningPhilosophers.jl"))
      using .DiningPhilosophers
      using DataFrames, CSV, Plots

      N = 5
      tmax = 50.0

      println("=== Классическая сеть (без арбитра) ===")
      net_classic, u0_classic, _ = build_classical_network(N)
      df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
      CSV.write(datadir("dining_classic.csv"), df_classic)
      dead = detect_deadlock(df_classic, net_classic)
      println("Deadlock обнаружен: $dead")
      plot_classic = plot_marking_evolution(df_classic, N)
      savefig(plotsdir("classic_simulation.png"))

      println("=== Сеть с арбитром ===")
      net_arb, u0_arb, _ = build_arbiter_network(N)
      df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
      CSV.write(datadir("dining_arbiter.csv"), df_arb)
      dead_arb = detect_deadlock(df_arb, net_arb)
      println("Deadlock обнаружен: $dead_arb")
      plot_arb = plot_marking_evolution(df_arb, N)
      savefig(plotsdir("arbiter_simulation.png"))

      === Классическая сеть (без арбитра) ===
      Deadlock обнаружен: true

      === Сеть с арбитром ===
      Deadlock обнаружен: false

[1]: ~/home/vvmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project/plots/arbiter_simulation.png"
```

Рисунок 4.15: Ноутбук `dining_philosophers_lit`

4.4 Реализация скриптов

4.4.1 Моделирование сети Петри

Цель: Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок. Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую.

Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри: классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock); модифицированная модель с арбитром, которая должна предотвращать deadlock.

```
using Pkg
Pkg.activate(joinpath(dirname(@__FILE__), "..", "project"))
Pkg.instantiate()

using DrWatson
using DataFrames, CSV
using Plots
gr()

include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers

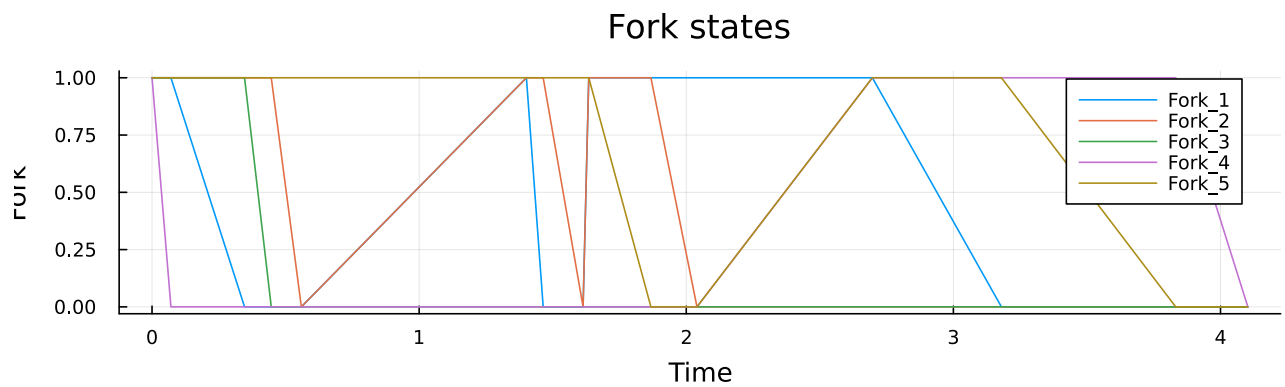
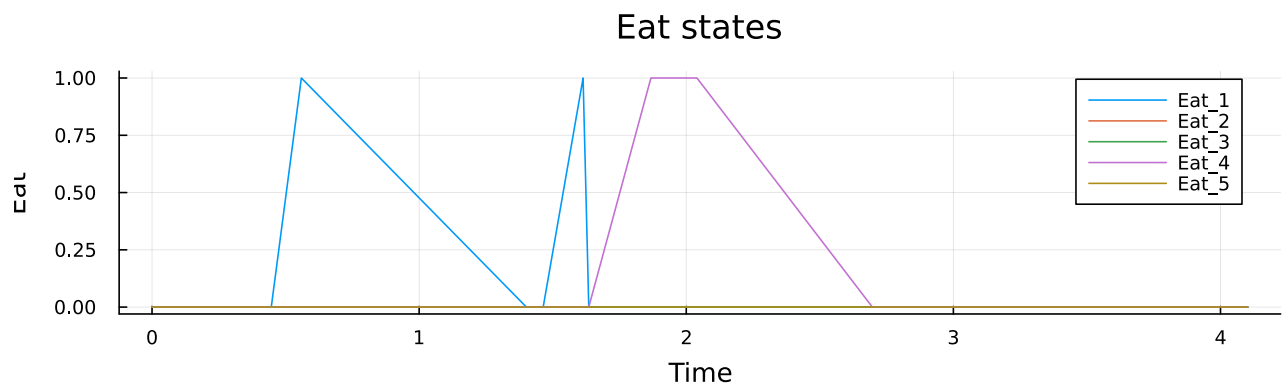
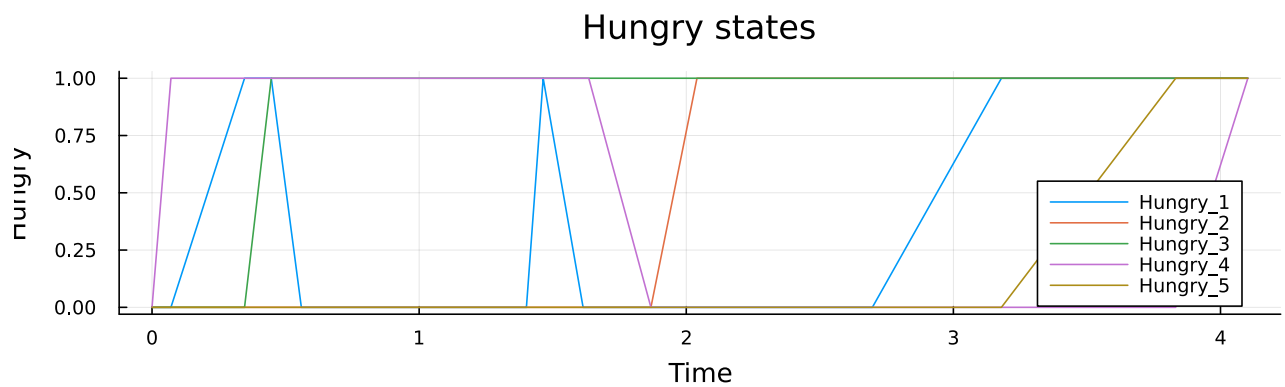
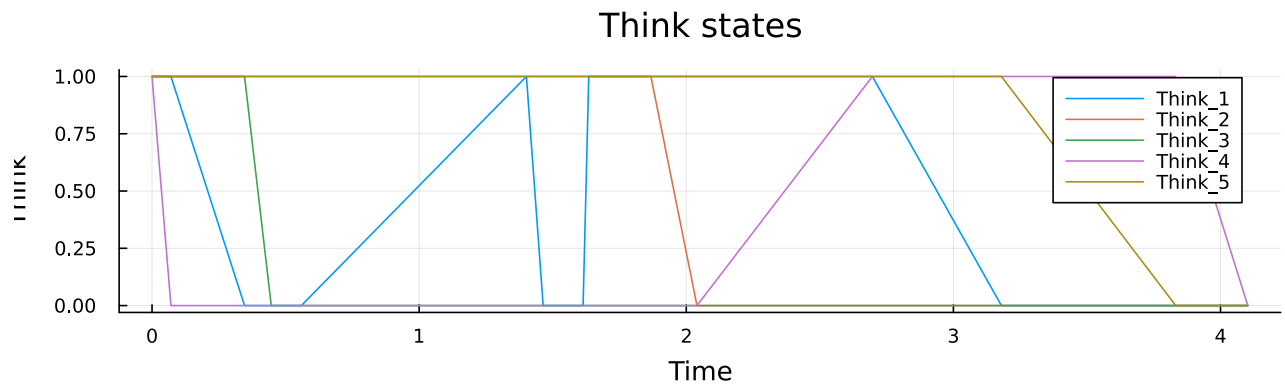
N = 5
tmax = 50.0

println("== Классическая сеть (без арбитра) ==")
net_classic, u0_classic, _ = build_classical_network(N)
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
```

```
CSV.write(datadir("dining_classic.csv"), df_classic)
dead = detect_deadlock(df_classic, net_classic)
println("Deadlock обнаружен: $dead")
plot_classic = plot_marking_evolution(df_classic, N)
savefig(plot_classic, plotsdir("classic_simulation.png"))

plot_classic
```

```
Activating project at `~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/proj
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true
Could not create decoration from factory! Running with no decorations.
```



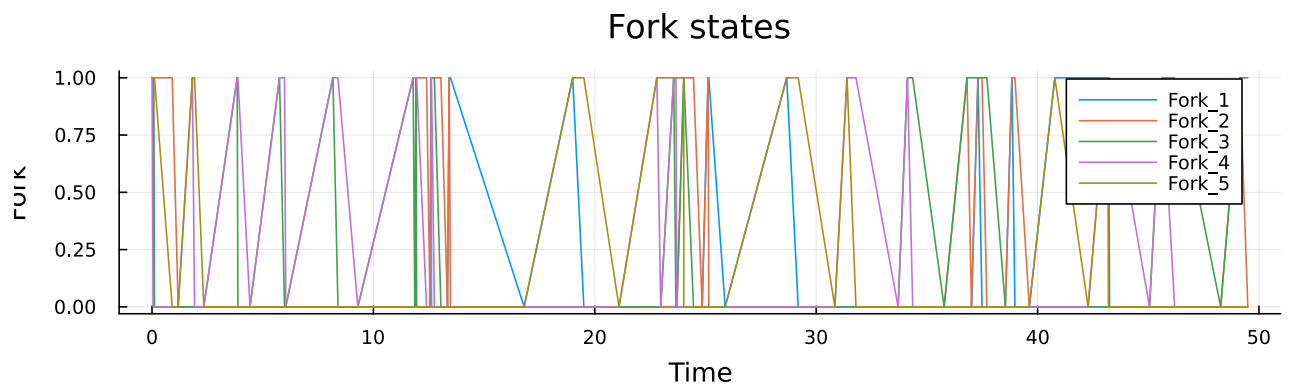
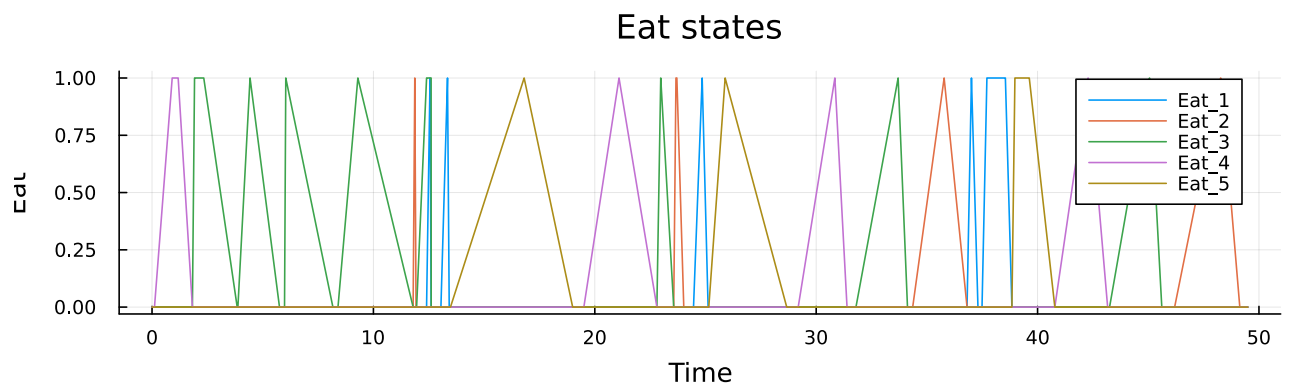
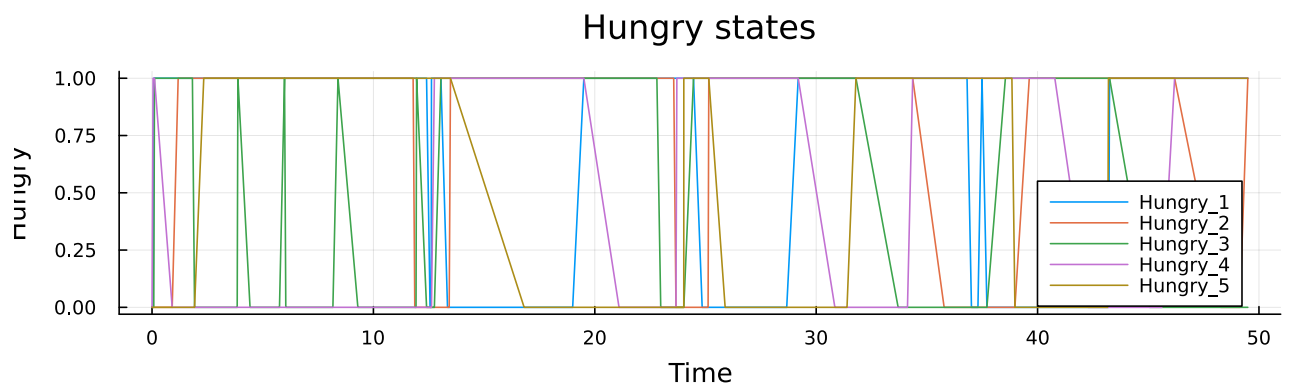
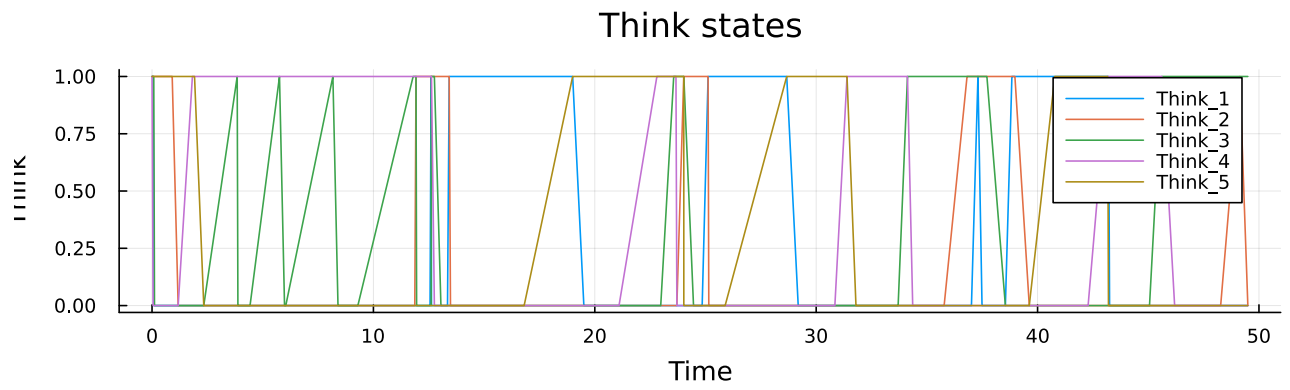
```
println("\n=== Сеть с арбитром ===")
net_arb, u0_arb, _ = DiningPhilosophers.build_arbiter_network(N)
```

```
df_arb = DiningPhilosophers.simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(datadir("dining_arbiter.csv"), df_arb)
dead_arb = DiningPhilosophers.detect_deadlock(df_arb, net_arb)
println("Deadlock обнаружен: $dead_arb")
plot_arb = DiningPhilosophers.plot_marking_evolution(df_arb, N)
savefig(plot_arb, plotsdir("arbiter_simulation.png"))
plot_arb
```

=== Сеть с арбитром ===

Deadlock обнаружен: false

Could not create decoration from factory! Running with no decorations.



4.4.2 Анимация сети Петри

Наглядная демонстрация динамики работы сети Петри во времени. Анимация позволяет увидеть, как меняется маркировка (фишки) в каждой позиции, и особенно наглядно показывает возникновение deadlock в классической модели.

```
using Pkg
Pkg.activate(joinpath(dirname(@__FILE__), "..", "project"))
Pkg.instantiate()

using DrWatson
using DataFrames, CSV
using Plots
gr()

using DrWatson
using Plots, Random

include(sourcedir("DiningPhilosophers.jl"))
using .DiningPhilosophers

N = 3
tmax = 30.0
net, u0, names = build_classical_network(N)

Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)

anim = @animate for row in eachrow(df)
```

```

u = [row[col] for col in propertynames(row) if col != :time]
bar(
    1:length(u),
    u,
    legend = false,
    ylims = (0, maximum(u0) + 1),
    xlabel = "Позиция",
    ylabel = "Фишки",
    title = "Время = $(round(row.time, digits=2))",
)
xticks!(1:length(u), string.(names), rotation = 45)
end
nothing
gif(anim, plotsdir("philosophers_simulation.gif"), fps = 2)
println("Анимация сохранена в plots/philosophers_simulation.gif")
nothing

```

```

Activating project at `~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project.toml`
WARNING: replacing module DiningPhilosophers.
WARNING: using DiningPhilosophers.build_classical_network in module Notebook conflicts with an existing definition
WARNING: using DiningPhilosophers.simulate_stochastic in module Notebook conflicts with an existing definition
WARNING: using DiningPhilosophers.detect_deadlock in module Notebook conflicts with an existing definition
WARNING: using DiningPhilosophers.plot_marking_evolution in module Notebook conflicts with an existing definition
Could not create decoration from factory! Running with no decorations.
[ Info: Saved animation to /home/vvmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/philosophers_simulation.gif
Анимация сохранена в plots/philosophers_simulation.gif

```

4.4.3 Сравнительный анализ

Сравнительный анализ двух моделей (с арбитром и без) по одному ключевому показателю — числу философов, находящихся в состоянии «Ест» (Eat_i).

Скрипт генерирует сводный график.

```
using Pkg

Pkg.activate(joinpath(dirname(@__FILE__), "..", "project"))
Pkg.instantiate()

using DrWatson
using DataFrames, CSV
using Plots
gr()

using DrWatson
using DataFrames, CSV
using Plots

df_classic = CSV.read(datadir("dining_classic.csv"), DataFrame)
df_arbiter = CSV.read(datadir("dining_arbiter.csv"), DataFrame)
N = 5

eat_cols = [Symbol("Eat_$(i)") for i = 1:N]

p1 = plot(
    df_classic.time,
    Matrix(df_classic[:, eat_cols]),
    label = [" $\Phi$  $(i)" for i = 1:N],
    xlabel = "Время",
    ylabel = "Ест (1/0)",
    title = "Классическая сеть",
)
p2 = plot(
```

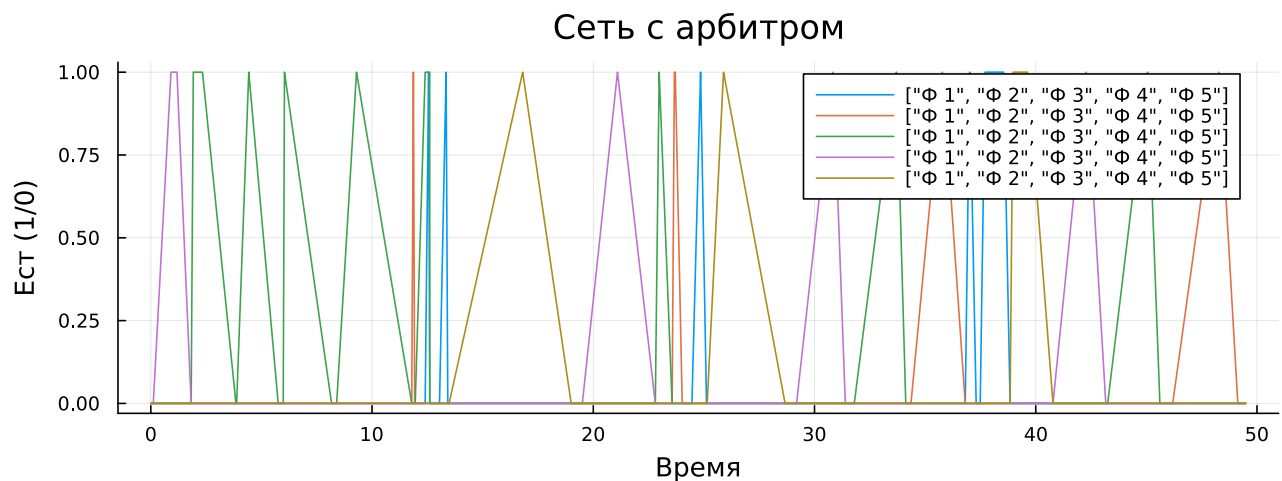
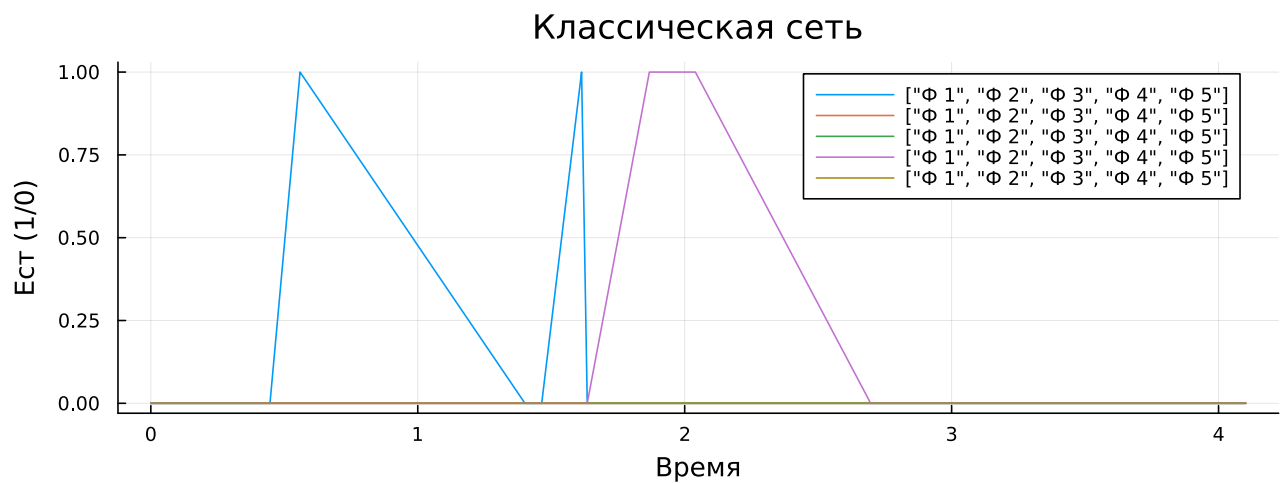
```

df_arbiter.time,
Matrix(df_arbiter[:, eat_cols]),
label = ["Φ $i" for i = 1:N],
xlabel = "Время",
ylabel = "Ест (1/0)",
title = "Сеть с арбитром",
)
p_final = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(p_final, plotsdir("final_report.png"))

p_final

```

Activating project at `~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project`
 Could not create decoration from factory! Running with no decorations.



```
println("Отчёт сохранён в plots/final_report.png")
```

Отчёт сохранён в plots/final_report.png

4.5 Анализ

Проведённое моделирование наглядно демонстрирует ключевую особенность классической задачи «Обедающие философы» — неизбежность взаимной блокировки при симметричной стратегии захвата вилок.

1. **Классическая сеть (без арбитра).** График `classic_simulation.png` (рис. 4.3) показывает, что после некоторого времени все философы оказываются в состоянии Hungry (голодны) и ни один не может перейти в состояние Eat (ест). Это подтверждается консольным выводом `Deadlock обнаружен: true` (рис. 4.1). Таблица `dining_classic.csv` фиксирует момент, после которого маркировка сети перестаёт изменяться — наступает deadlock.
2. **Сеть с арбитром.** Введение дополнительной позиции `Arbiter` с начальным количеством фишек $N-1$ кардинально меняет динамику. График `arbiter_simulation.png` (рис. 4.4) демонстрирует непрерывную смену состояний: философы поочерёдно думают, голодают и едят, и по крайней мере один из них всегда находится в состоянии `Eat`. Консольный вывод `Deadlock обнаружен: false` подтверждает работоспособность арбитра как механизма синхронизации.
3. **Сравнительный анализ.** График `final_report.png` (рис. 4.9) сводит воедино динамику состояния `Eat` для обеих моделей. Видно, что в классической сети линии всех философов падают до нуля и остаются на нуле, тогда как в сети с арбитром наблюдаются регулярные колебания. Это служит количественным доказательством эффективности выбранного метода предотвращения deadlock.

4. Анимация. GIF-анимация (рис. 4.8) позволяет визуально проследить за перемещением фишек в классической модели и увидеть момент «застывания» системы, когда все переходы становятся неактивными.

Таким образом, полученные результаты полностью согласуются с теоретическими ожиданиями и подтверждают, что даже минимальная модификация (ограничение числа одновременно обедающих философов) способна исключить взаимную блокировку.

5 Выводы

В результате выполнения лабораторной работы №5 «Аппарат сетей Петри. Моделирование задачи „Обедающие философы“» были решены все поставленные задачи:

1. Построены две сети Петри для пяти философов: классическая (приводящая к deadlock) и с арбитром (предотвращающая deadlock).
2. Реализовано стохастическое моделирование с использованием алгоритма Гиллеспи; подтверждено возникновение взаимной блокировки в классической модели и её отсутствие в модели с арбитром.
3. Получены и проанализированы графики эволюции маркировок, сводный график состояния «Ест», а также создана анимация, наглядно иллюстрирующая динамику системы.
4. Код оформлен в стиле литературного программирования с использованием `Literate.jl`. Сгенерированы производные форматы: чистые скрипты, Jupyter Notebook и Quarto-документация.
5. Подготовлен отчёт в формате Quarto, содержащий все необходимые разделы, перекрёстные ссылки, библиографию и встроенные результаты моделирования. Отчёт успешно скомпилирован в форматы PDF, HTML и DOCX.
6. Рабочий репозиторий размещён на GitHub с соблюдением соглашения об именовании Denote, семантического версионирования и общепринятых коммитов.

Итог: Изучен математический аппарат сетей Петри как инструмент для анализа параллельных процессов. На примере задачи «Обедающие философы» показано, как формальная верификация позволяет выявить проблему deadlock и предложить простое, но эффективное решение. Полученные навыки могут быть применены при проектировании и анализе распределённых систем, баз данных и многопоточных приложений.

Список литературы

1. *Coffman Jr E. G., Elphick M. J., Shoshani A.* System deadlocks // ACM Computing Surveys (CSUR). — 1971. — Т. 3, № 2. — С. 67—78.
2. *Dijkstra E. W.* Hierarchical ordering of sequential processes // Acta Informatica. — 1971. — Т. 1, № 2. — С. 115—138.
3. *Petri C. A.* Kommunikation mit Automaten. — Schriften des Rheinisch-Westfälischen Institutes für instrumentelle Mathematik an der Universität Bonn, 1962.